

El Motor del Ecosistema Java

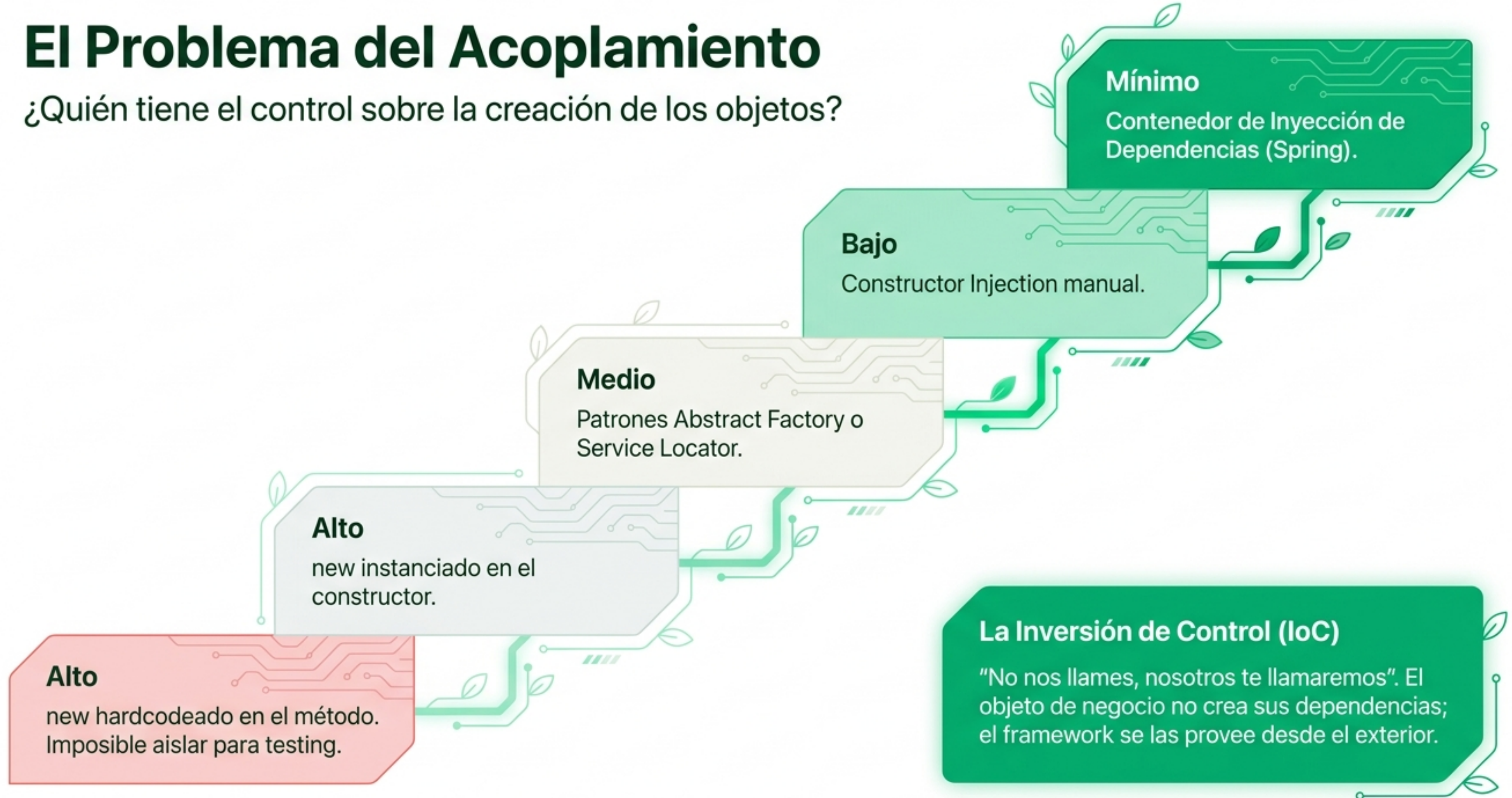
Módulo 4: Arquitectura y Conexiones de Spring Boot

Descubriendo la Inversión de Control, la Inyección de Dependencias y el ensamblaje automático de módulos.



El Problema del Acoplamiento

¿Quién tiene el control sobre la creación de los objetos?



Evolución a Convenciones Sensatas

De la configuración manual al arranque instantáneo.

Spring Framework

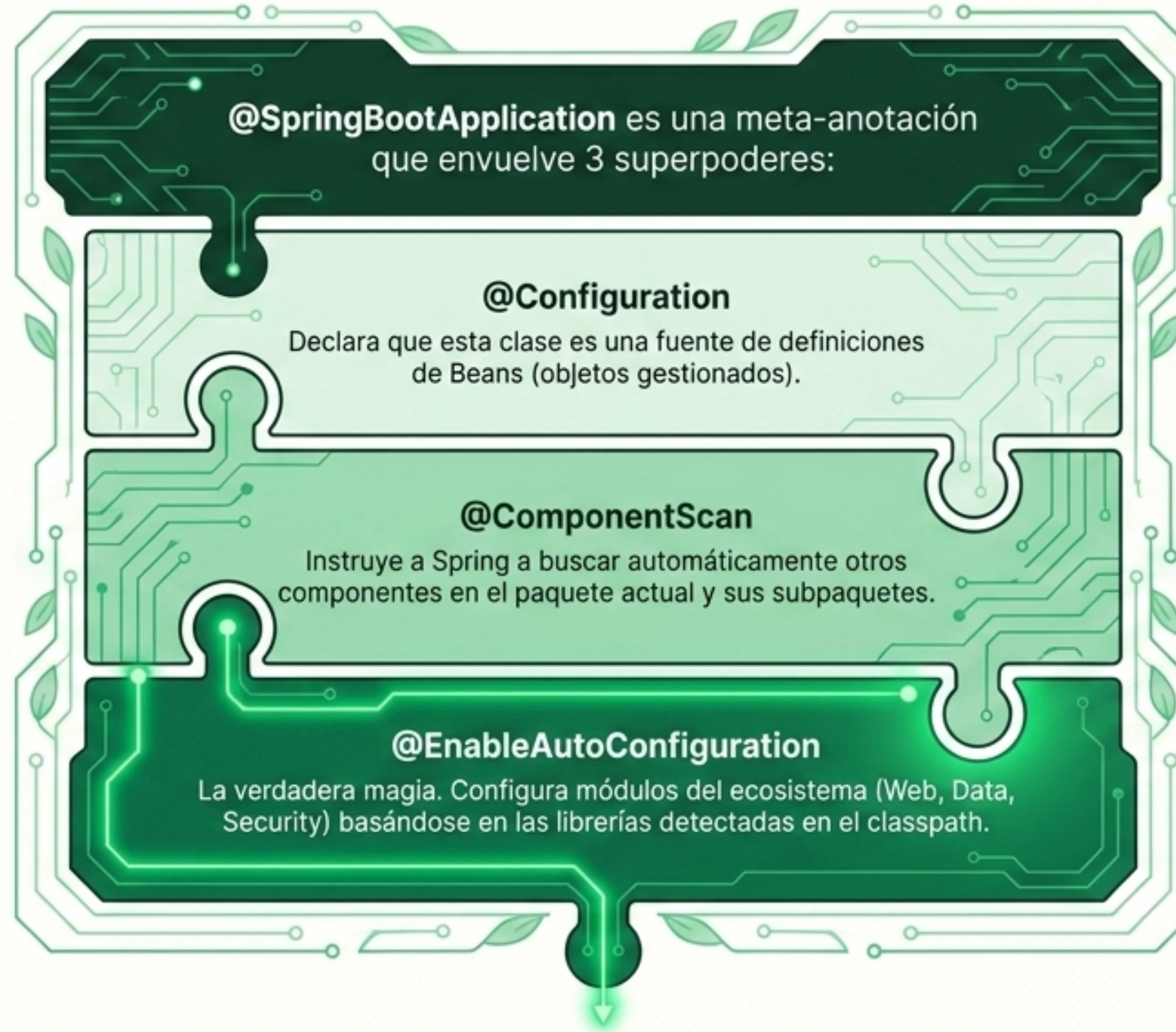
- **Configuración:** Explícita y verbosa (XML o clases de configuración extensas).
- **Arranque:** Configuración manual compleja del servidor.
- **Dependencias:** Gestión manual versión por versión.
- **Servidor:** Requiere instalación externa de Tomcat/Jetty.
- **Filosofía:** Extremadamente flexible, pero configúralo tú mismo.

Spring Boot

- **Configuración:** Automática por convenciones predefinidas.
- **Arranque:** Un simple método `main()` con un click.
- **Dependencias:** Starter packs unificados.
- **Servidor:** Tomcat embebido directamente en el `.jar`.
- **Filosofía:** Opiniones sensatas listas para producción.

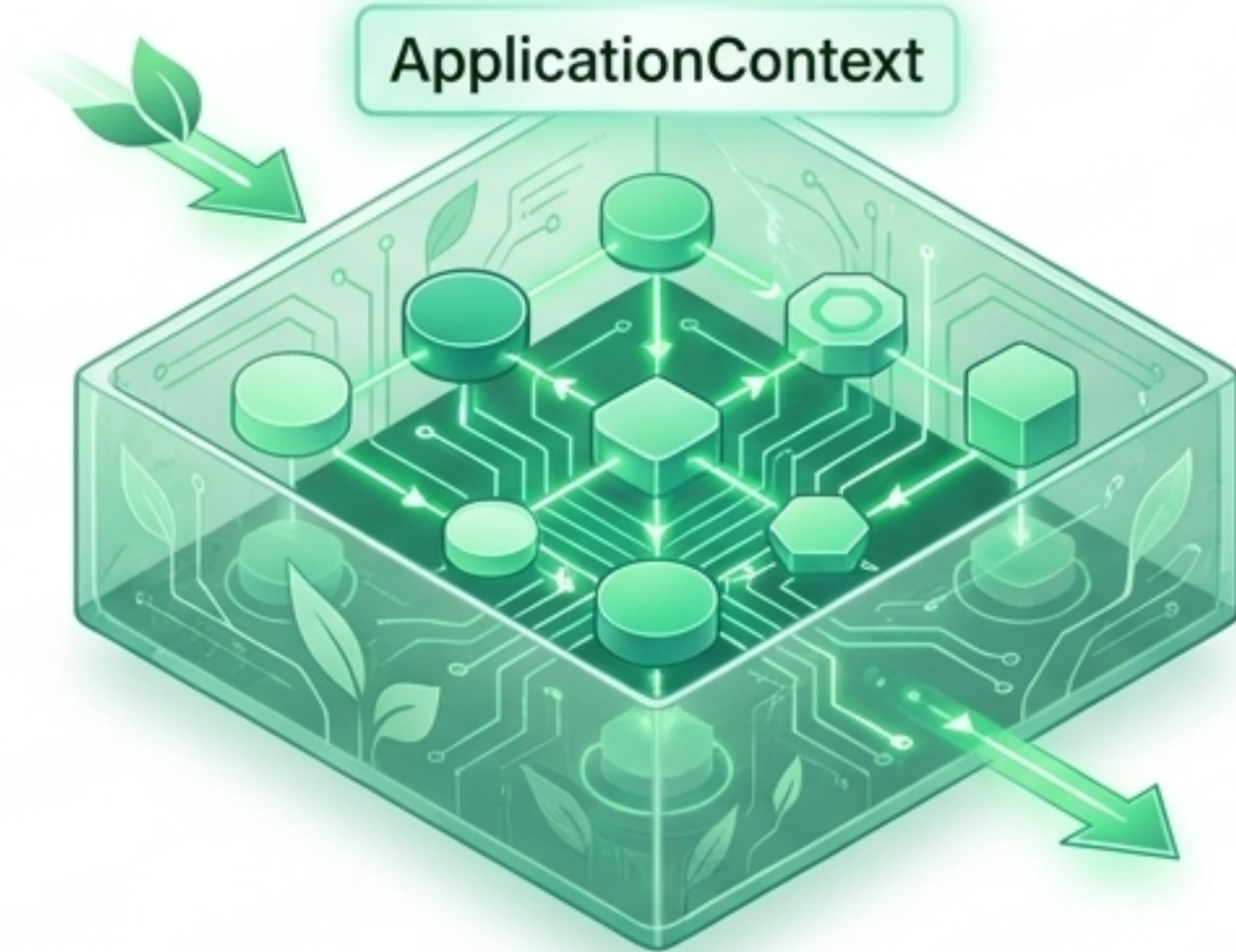
La Anatomía del Arranque

El poder oculto detrás de @SpringBootApplication.



El Cerebro: ApplicationContext

El contenedor central que da vida y gestiona todo el ecosistema.



¿Qué hace el ApplicationContext?

Es la fábrica y registro de Spring. Escanea el código, crea las instancias (Beans), resuelve sus dependencias cruzadas y los almacena listos para su uso.

1. Singleton por defecto

Mantiene UNA sola instancia compartida de cada Bean en todo el contexto.

2. Ciclo de Vida

Controla el nacimiento (@PostConstruct) y la muerte (@PreDestroy) de los objetos.

3. Cableado Invisible

Injecta los objetos automáticamente donde las firmas de los constructores los requieran.

Semántica por Capas

Organizando el dominio mediante estereotipos arquitectónicos.



El Estándar de Inyección

Por qué la inyección por constructor es la mejor práctica absoluta.

Avoid: Field Injection

```
@Service
public class PagoService {
    @Autowired // Acoplamiento oculto
    private Notificador notificador;
}
```

Recommended: Constructor Injection

```
@Service
public class PagoService {
    private final Notificador notificador;

    // Spring inyecta esto automáticamente
    public PagoService(Notificador notificador) {
        this.notificador = notificador;
    }
}
```

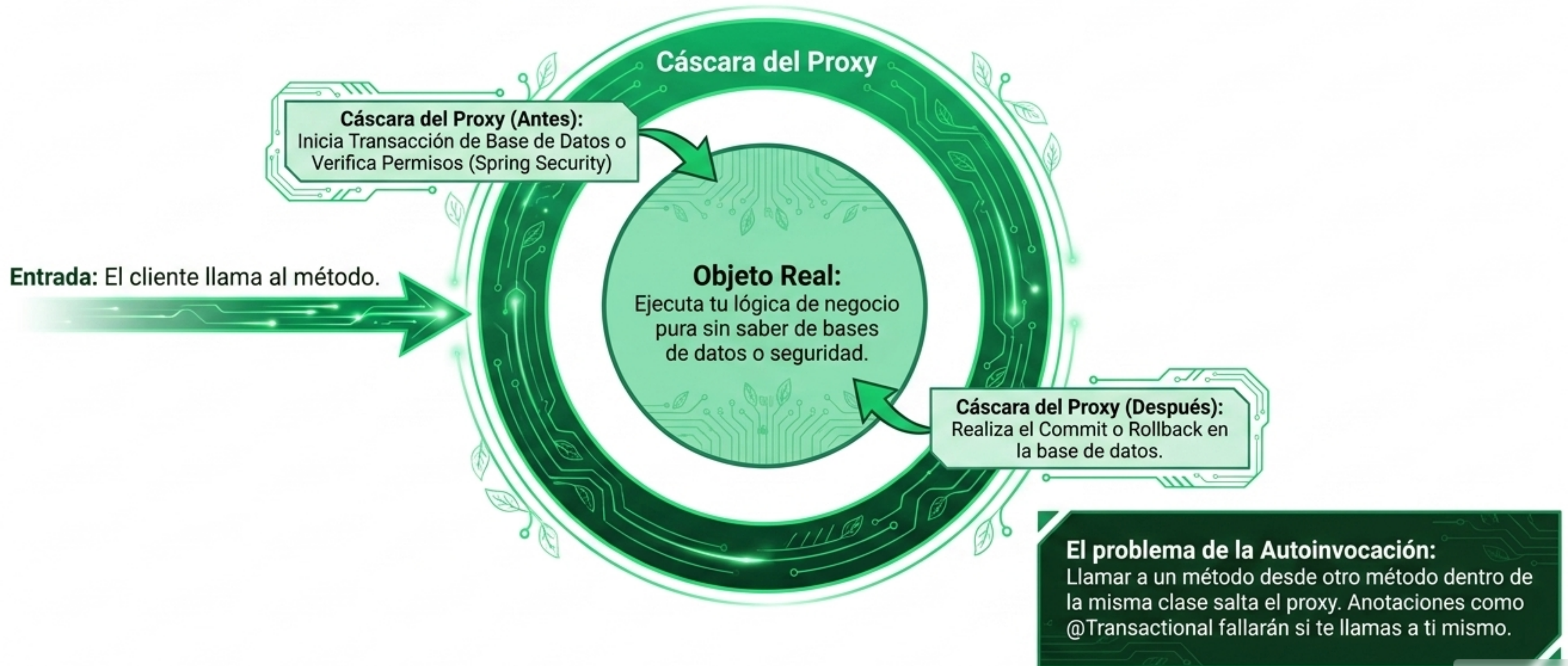
Ventajas del Constructor:

Garantiza la inmutabilidad (final), hace las dependencias explícitas, previene NullPointerExceptions y permite testear la clase aislando Spring mediante mocks (Mockito).

La Magia Oculta: Proxies

Cómo intercepta Spring las llamadas para inyectar el ecosistema

Cuando inyectas un Bean, no recibes el objeto original, recibes un Proxy dinámico generado por Spring (JDK/CGLIB).



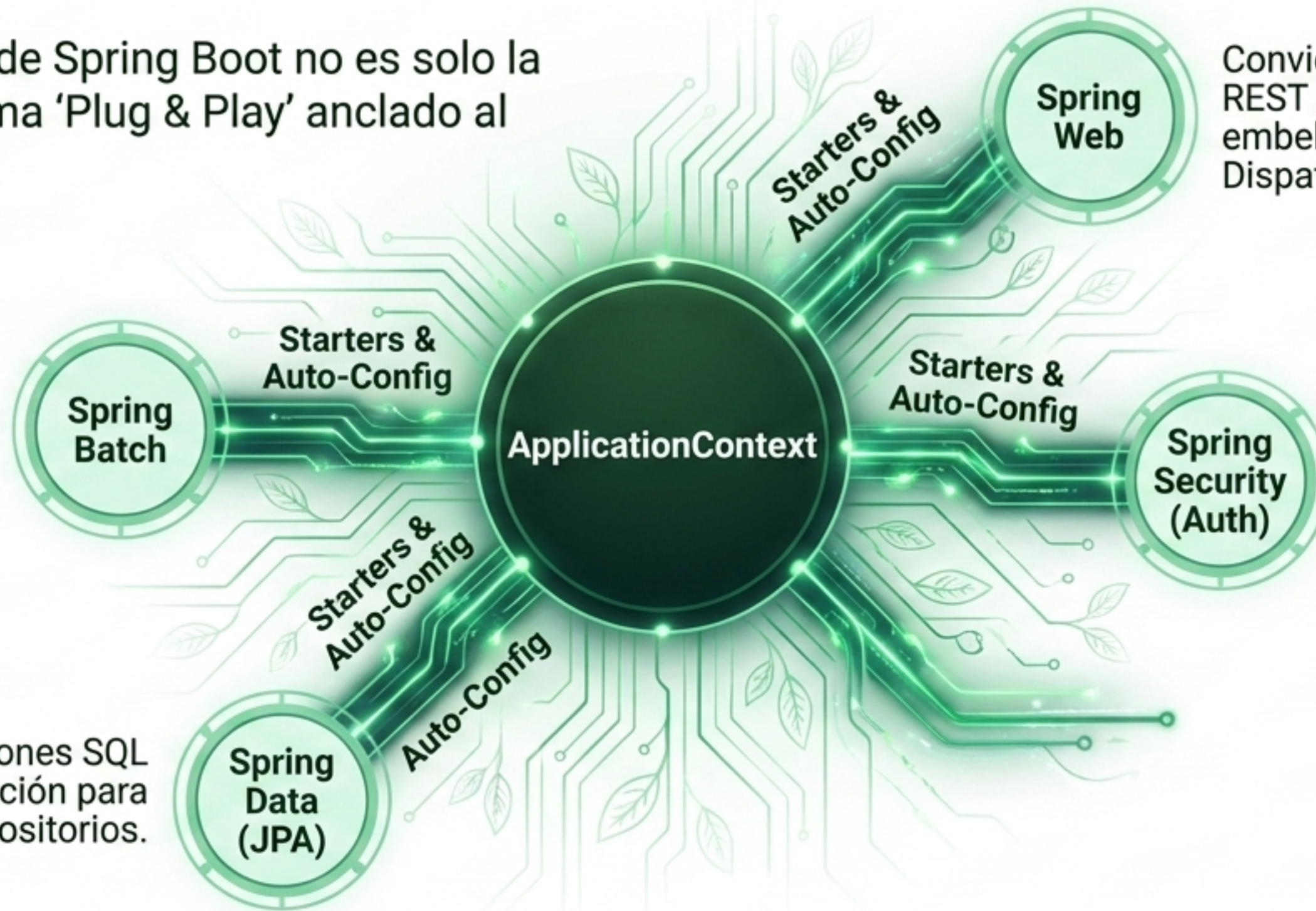
El Hub del Ecosistema

Cómo se conectan los módulos externos a la placa base.

El verdadero poder de Spring Boot no es solo la IoC, es su ecosistema 'Plug & Play' anclado al contenedor central.

Toma las conexiones de base de datos ya configuradas en el contexto y orquesta trabajos masivos en segundo plano.

Genera implementaciones SQL en tiempo de ejecución para tus repositorios.

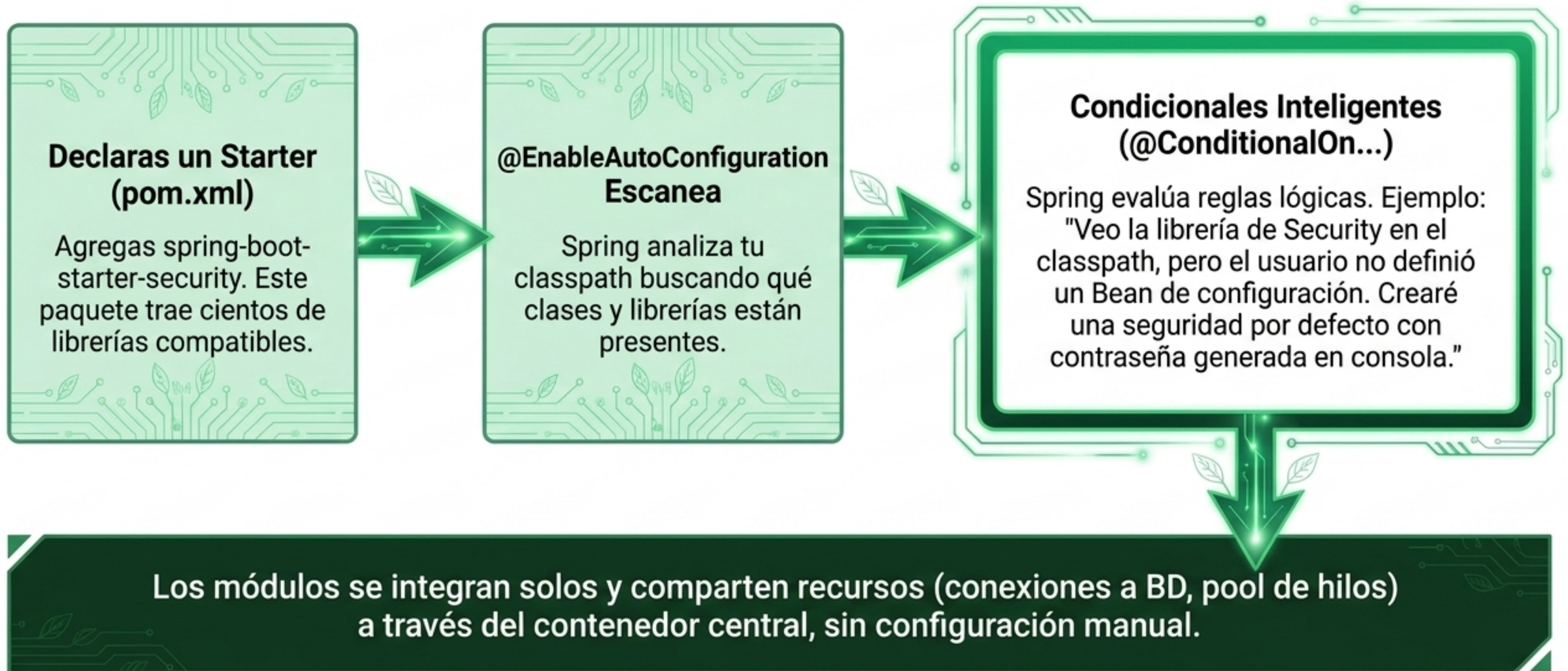


Convierte tus beans en APIs REST usando el Tomcat embebido y el DispatcherServlet.

Se inyecta como un filtro HTTP antes de tus Controllers, interceptando las peticiones gracias a los Proxies.

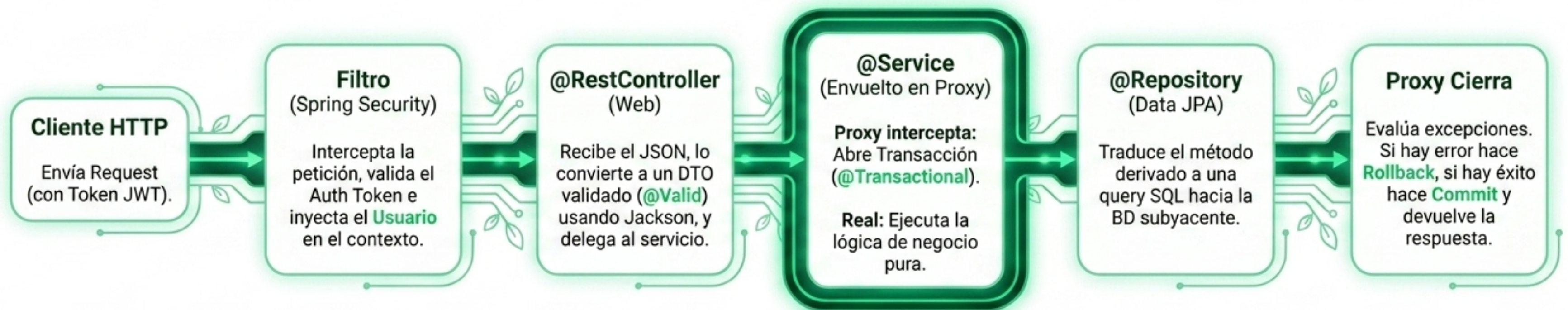
El Motor de Conexión: Autoconfiguración

¿Cómo sabe Spring Boot exactamente qué ecosistema levantar?



Mapa Arquitectónico Completo

El ciclo de vida de una petición a través de todo el ecosistema.



Desacoplamiento Extremo: Eventos

Evitando el código espagueti con Application Events.

Cuando los módulos necesitan comunicarse, no siempre deben inyectarse mutuamente. Spring implementa el patrón Observer de forma nativa.



Emite un evento puro de dominio:
`publisher.publishEvent(new UsuarioCreadoEvent(user))`

Arquitectura Limpia: El servicio original no necesita saber que el sistema de seguridad o el módulo Batch existen.

Receptor 1 (Módulo Auth)

@EventListener
(Asigna roles por defecto).

Receptor 2 (Módulo Batch)

@EventListener
(Encola un trabajo en segundo plano para procesar sus datos).

Receptor 3 (Auditoría)

@TransactionalEventListener
(Registra en base de datos solo si la transacción principal fue exitosa).

Adaptabilidad del Entorno: Propiedades y Perfiles

Cómo el ecosistema muta entre Dev, Test y Prod.

Spring Boot unifica la configuración de TODO el ecosistema (puertos web, dialectos SQL, llaves JWT) en un solo lugar y las resuelve en este orden de precedencia:

Best Practice

@ConfigurationProperties: En lugar de inyectar variables sueltas con @Value, usa esta anotación para mapear configuraciones fuertemente tipadas a clases Java validadas.



El Patrón del Ecosistema

Principios tácticos para dominar Spring Boot.



Deja el control al

Contexto: No uses new para objetos de dominio. Deja que el ApplicationContext ensamble tu sistema.



Inyección Inmutable:

Prioriza la inyección por constructor. Haz tus dependencias final y claras.



Respetar las Fronteras:

Mantén la separación estricta entre @Controller, @Service y @Repository.



Aprovecha los Starters:

No reinventes la rueda. Usa autoconfiguración para integrar Security, Data y Batch al instante.



Cuidado con el Proxy:

Recuerda que la autoinvocación interna salta las transacciones y la seguridad.



Desacopla Efectos

Secundarios: Usa @EventListener para lógicas paralelas (emails, auditorías) sin ensuciar tu negocio.



Has ensamblado el motor.
Fin del Módulo 4.

Próximo paso: Módulo 5 - APIs REST

Estás listo para usar el entorno de Spring Boot y el DispatcherServlet para exponer tus contenedores al mundo exterior mediante endpoints seguros y validados.